

DocuTrust: Enterprise-Grade Corrective RAG (CRAG) System

Technical Design and User Manual

1. Executive Summary

DocuTrust is a self-correcting enterprise Retrieval-Augmented Generation (RAG) system. Typical RAG pipelines suffer from hallucinations, database search noise, and incomplete internal corpora. DocuTrust addresses these weaknesses using a Corrective RAG (CRAG) architecture that incorporates local vector search, Cross-Encoder relevance grading, query re-writing, allowlisted web fallback searching, and strict post-generation citation validation.

2. System Architecture & Components

The DocuTrust system consists of five tightly integrated layers, executing in a linear-with-conditional-branch pipeline managed via FastAPI and LangGraph:

2.1 Ingestion Layer

During ingestion, uploaded documents (PDFs, Word documents, or plain text) are parsed and split into overlapping semantic paragraphs. Each chunk is assigned metadata including its parent document ID, section path, and source details. The raw text and metadata are saved to MongoDB, while the text is embedded using OpenAI's 'text-embedding-3-small' model and stored in a local FAISS index.

2.2 Retrieval Layer

For every incoming query, the question is vectorized using the same OpenAI embeddings model. A similarity search is performed against the FAISS index to retrieve the top K nearest chunks, which are then hydrated with their text content and metadata from MongoDB.

2.3 Cross-Encoder relevance grading

Rather than relying solely on vector similarity, DocuTrust routes all retrieved chunks through a local Cross-Encoder model ('cross-encoder/ms-marco-MiniLM-L-6-v2'). The Cross-Encoder performs self-attention over the query and text together, assigning a normalized score from 0.0 to 1.0. Chunks scoring below the grader threshold (default 0.55) are classified as irrelevant and discarded.

2.4 Corrective Web Fallback

If no chunks pass the Cross-Encoder grading threshold, the internal corpus is deemed insufficient. The query is automatically sent to the LLM (gpt-4o-mini) to be rewritten into optimized search engine terms. A search is executed on the open web using DuckDuckGo, restricted to high-quality allowed domains (such as sec.gov, nist.gov, and europa.eu). These web search results are formatted into chunks and used as context.

2.5 Generation & Gating Layer

The LLM generates a response using the compiled context and is instructed to cite references as '[chunk_id]'. The backend parses the LLM output; if the answer lacks valid citation markers, contains hallucinated citations, or prefixes with 'REFUSE', the system blocks the response and returns a formal refusal, ensuring zero hallucinations.

3. Setup and Configuration

Configure the application via a '.env' file located in the project root. Below is a sample configuration template:

```
OPENAI_API_KEY=your-openai-api-key
MONGO_URI=mongodb://localhost:27017
MONGO_DB=docutrust
VECTOR_BACKEND=faiss
FAISS_INDEX_PATH=./data/faiss
OPENAI_CHAT_MODEL=gpt-4o-mini
OPENAI_EMBED_MODEL=text-embedding-3-small
GRADER_MODEL=cross-encoder/ms-marco-MiniLM-L-6-v2
GRADER_THRESHOLD=0.55
RETRIEVAL_K=8
WEB_DOMAINS_ALLOWLIST=sec.gov,europa.eu,nist.gov,iso.org
WEB_MAX_RESULTS=5
```

4. Local Execution & Ingestion Commands

Follow these command-line instructions to run and populate the project locally:

- Start the Backend API Server:

```
.venv/bin/uvicorn app.main:app --reload --port 8000
```

- Seed the database and ingest demo files:

```
.venv/bin/python -m scripts.seed_corpus
```

- Start the Streamlit Frontend application:

```
.venv/bin/streamlit run ui/streamlit_app.py
```

- Execute the testing suite:

```
.venv/bin/pytest
```